

Project 13 (Bonus) — CodeGuard-Multi-Agent-PR-Review-Security-Style-And-Test-Coverage-In-One-Pass

Title: CodeGuard Multi Agent PR Review Security Style And Test Coverage In One Pass

Type: Multi-agent (4 agents: 1 Supervisor + 3 reviewers) · **Stack:** Python, LangChain, LangGraph, RAG, FastAPI, Docker · **Level:** Advanced (a test of what you've learned) **Tagline:** A supervisor-led team of three specialist reviewers — Security, Style, and Test-Coverage — that turns a raw code diff into one compiled review comment.

Not part of the numbered Doc01-19 sequence — this project doesn't have its own document teaching new ideas, because it doesn't need one. Every skill it uses was already taught in Docs 01-19 and proven in Projects 1-5. Full detail lives here, and only here.

Jump to: [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#)

Why This Project Exists

Projects 1-5 all build toward the same area (content research and publishing), so each one could focus on exactly one new design idea, without a new problem area getting in the way. That's good teaching, but it leaves you with only one story for a portfolio or interview. **This project exists to prove the design generalizes** — the same multi-agent patterns (Doc11), the same RAG (Doc08), the same production work (Doc12-13, 19), used on a completely different problem: reviewing code, instead of writing content.

The Story — what you're actually building

Imagine a pull request lands, and before any human reviewer opens it, three specialists already looked it over. One checked for security problems — secrets left in code, obvious unsafe patterns. One checked it against your team's actual style guide — not generic advice, your real document. One checked whether the changed code actually has tests covering it. A fourth person — the lead — reads all three reports, throws out anything that overlaps or doesn't really matter, and writes one clear comment: here's what's actually worth your attention. That's CodeGuard: the same supervisor-plus-specialists shape as ContentForge (Project 4), pointed at a completely different problem.

The point of building this second system isn't the code review itself — it's proving that what you learned building ContentForge wasn't a one-off trick that only works for "research then write" pipelines. A Supervisor that routes work to the right specialist, and specialists that stay narrow and grounded (the Style reviewer searches your real style guide with RAG instead of guessing from general training knowledge) — that shape shows up again and again once you start looking for it: content moderation, compliance checks, multi-criteria approvals. If you can rebuild it here, on a different problem, with much less guidance than Projects 1-5 gave you, that's the real proof it generalizes.

That's also why this project is deliberately lighter on hand-holding than Projects 1-4. You've now built four multi-agent systems. Step 1 through Step 3 give you the shape of each step — what it needs to prove — but leave the smaller sub-steps for you to figure out, the way a real assignment at a job would. The hints below exist if you get stuck, but try harder here before reaching for them than you did on earlier projects — that's the actual exercise.

What you're actually building, in one line: a Supervisor-led team of three specialist reviewers — Security, Style, and Test-Coverage — that turns a raw code diff into one compiled review comment.

Why this needs to exist: reviewing a pull request well needs real judgment calls a fixed rule set can't make — whether the tests that exist actually cover the meaningful cases, or whether a change fits your team's real style intent and not just its letter — and a human reviewer's time is better spent once the easy, repeatable checks are already done.

When you'd reach for this at a real job: a review bot wired into CI/CD, or a first-pass check that runs before a human reviewer even opens a pull request, on any team where secrets, style, and test coverage are worth catching automatically.

How it works, mechanically: the Supervisor sends the diff's changed lines to whichever reviewers actually apply, each reviewer works on its own narrow job, and the Supervisor merges their findings into one sorted, de-duplicated comment.

Why not just do it some simpler/different way: why not run a single linter or static-analysis tool instead of LLM agents? You should, for what static analysis is actually good at — syntax problems, obvious secret patterns. But static analysis can't judge whether the tests that exist actually cover the meaningful cases, or whether code matches your team's real style intent instead of just its letter — those need judgment, not just pattern matching, which is exactly why this project uses reviewer agents instead of stopping at a linter.

Why This Project Is Good for Your Portfolio

- **What problem it solves:** automates the boring, easy-to-skip parts of reviewing a pull request — checking for secrets left in code, obvious security mistakes, style-guide problems, and changed code with no tests — before a human reviewer's time gets spent on it.

- **Why it matters:** it's a second, different multi-agent system in your portfolio, in an area (developer tools) that any technical interviewer immediately understands — and it proves the skills from Projects 1-5 weren't just memorized for one specific pipeline.
- **When you'd build something like this at a real job:** review bots built into CI/CD, internal code-quality tools, or a first-pass check before a human reviewer even looks at a PR — genuinely useful even on a small team.
- **How it's built:** a Supervisor sends a diff to whichever reviewers actually apply (skip Security on a docs-only change, skip Style on a config file), each reviewer works on its own, and the Supervisor puts their findings together into one final comment.

Problems you'll likely run into, and how to fix them:

Problem	Fix
The Security reviewer flags something in code that didn't actually change (an old issue, not caused by this diff)	Limit every reviewer's input to <i>only</i> the diff's added/changed lines, not the whole file — the same idea as Doc08's chunking: only look at what's actually relevant
The Style reviewer disagrees with your actual style guide, because it's answering from general training knowledge instead of your specific guide	This is exactly RAG's job (Doc08) — search your actual style-guide document, don't let the reviewer freelance from what it already "knows"
The Supervisor runs all 3 reviewers on every diff, no matter what, wasting money	Route based on which files actually changed (following Doc11's routing discipline) — a <code>.md</code> -only diff doesn't need a security check
The compiled final review is just the 3 reviewers' output stuck together, hard to read	The Supervisor's compiling step is a real task on its own — remove overlapping findings, sort by how serious they are, and write one clear comment, not a copy-paste of three separate reports

Setup (do this once, before Step 1)

```
cd project_13_codeguard
python -m venv .env
source .env/bin/activate
pip install openai langchain langchain-openai langgraph chromadb fastapi "uvicorn
pip freeze > requirements.txt
```

Same `.env` setup as every other project (`OPENAI_API_KEY` , `LOG_LEVEL`). You'll also need a small style-guide document (write 1-2 pages yourself, or use a real one from a project you know) to build the RAG knowledge base from.

Don't have a private repo with real pull requests handy? Any public GitHub repo works fine for practice — clone one with recent PRs, or use one of your own.

Plan Before You Code

Same process as every project (see Doc15): Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks. This time, **do it with less guidance from this document** — that's the whole point. If you get stuck on the process itself (not the AI/LangGraph mechanics), go back to [15_five_projects_index](#) and [16_system_design_architecture](#).

How To Build This — Step by Step

On purpose, less detailed than Projects 1-5 — you've built four multi-agent systems by now. This document trusts you to break each step into your own smaller steps, the way Projects 1-5 did it for you.

Step 1 — One Reviewer, Working as a Plain Chain on a Made-Up Diff

Project: CodeGuard-Multi-Agent-PR-Review-Security-Style-And-Test-Coverage-In-One-Pass — Step 1 of 4: One Reviewer, Working as a Plain Chain on a Made-Up Diff

What this step does: proves you can figure out this project's own version of "Step 1" from a one-line description now, without me spelling out smaller steps — the way Doc04/05's Core Concepts, used by you, would suggest. **Why this step matters:** the actual skill this whole bonus project is testing is starting from a one-line description and building your own first step — proving that on the smallest possible piece here, before anything else depends on it, is what makes the rest of the project trustworthy. **What's new:** nothing exists yet — this is the start. **What stays the same going forward:** whatever chain you build here becomes the reused core of your Security (or whichever you pick first) reviewer agent. **When you'll hit this for real:** any time you're handed a new problem domain and have to figure out your own first step — which is most of a real job, once you're past following tutorials. **Read first (if needed):** [05_langchain_fundamentals Core Concepts](#), [06_tools_function_calling Core Concepts](#) if your reviewer needs a tool (like a text-search secret scanner as a real tool, not just something you prompt for).

Pick one reviewer (Security is a good start — clear, checkable rules) and get it answering correctly on 2-3 made-up example diffs before anything else exists. Include at least one diff that's clean on purpose, not just diffs with a planted problem — a reviewer you've only ever tested against bad input will happily flag something in code that's actually fine.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Step 2 — That Reviewer, Turned Into an Agent; Add the Style Reviewer With RAG

Project: CodeGuard-Multi-Agent-PR-Review-Security-Style-And-Test-Coverage-In-One-Pass — Step 2 of 4: That Reviewer, Turned Into an Agent; Add the Style Reviewer With RAG

What this step does: brings in review grounded by search (your style guide), and proves two separately-built reviewers can exist side by side without getting in each other's way. **Why this step matters:** an ungrounded Style reviewer answering from general training knowledge instead of your real document would look correct right up until it confidently cites a rule that isn't actually yours — this step is what makes that failure mode checkable instead of invisible. **When you'll hit this for real:** any review or QA tool checking against a written standard (style guide, compliance rules, brand voice) — the model needs to check against your real document, not its own training knowledge. **Read first (if needed):** [08_rag Core Concepts](#).

Build the Style reviewer against a real, searched style guide (not the model's general knowledge — this is the whole point, see the problems/fixes table above). Test both reviewers separately, against the same set of example diffs. Write at least one style rule into your guide that's specific or unusual enough a model wouldn't invent it from general training knowledge on its own — a generic rule any model would get right regardless proves nothing about whether retrieval is actually working.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Step 3 — Add the Test-Coverage Reviewer + Supervisor Routing (final: 4 agents)

Project: CodeGuard-Multi-Agent-PR-Review-Security-Style-And-Test-Coverage-In-One-Pass — Step 3 of 4: Add the Test-Coverage Reviewer + Supervisor Routing

What this step does: finishes the team and adds the coordination layer — routing, compiling results, and the actual multi-agent value. **Why this step matters:** 3 separately-correct reviewers stuck together is not the same thing as one useful review — this step is where the actual value of a multi-agent system over 3 standalone tools either shows up or doesn't, which is the whole thing this project exists to prove. **When you'll hit this for real:** any "combine several specialist opinions into one clear recommendation" system — code review, content moderation, multi-criteria approval workflows all have this exact shape. **Read first (if needed):** [11_multi_agent_systems Core Concepts](#).

Build the Test-Coverage reviewer, then the Supervisor: routing based on the diff (which reviewers actually apply), a compiling step that merges findings into one clear review, and — reusing Doc11's revision-loop idea if you want to go further — an optional pass where the Supervisor can ask a reviewer to double-check an unclear finding before finishing. If you build that optional pass, give it a hard round limit the same way Project 4's Writer ↔ Reviewer loop needed one — an "ask for clarification" loop with no limit is the exact same infinite-loop risk in a new shape.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Step 4 — Production Wrap (optional, reuses Project 5's steps completely)

Project: CodeGuard-Multi-Agent-PR-Review-Security-Style-And-Test-Coverage-In-One-Pass — Step 4 of 4: Production Wrap

What this step does: wraps the finished 4-agent system in the exact same production layer Project 5 already built — FastAPI, a database, Docker, and a test-gated release process — proving that layer generalizes too, not just the agent design. **Why this step matters:** it's the difference between a portfolio project you can only describe and one you can hand someone a real link to — and doing it a second time, by reusing Project 5 almost unchanged, is what actually makes Doc12/13/19's patterns stick as reusable skills instead of one-off code you happened to write once. **When you'll hit this for real:** any portfolio project you want a real, runnable link for instead of just source code — this is what makes CodeGuard something you can point an interviewer at, not just describe.

If you want this as a real, portfolio-runnable API: wrap it in FastAPI, add a test suite (a set of example diffs with known-correct findings), and containerize it — literally repeat Project 5's Steps 1-4, on this codebase instead of ContentForge's. No new ideas here — just repetition, which is the fastest way to make Doc12/13/19 stick permanently. Include at least one eval task that specifically checks Step 3's routing (e.g. a docs-only diff correctly skipping the Security reviewer) — a suite that only checks each reviewer's accuracy could still pass even if routing were completely broken.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Final Deliverable: CodeGuard-Multi-Agent-PR-Review-Security-Style-And-Test-Coverage-In-One-Pass — a Supervisor-led team of 3 specialist reviewers that turns a code diff into one compiled, sorted review comment, grounded in your actual style guide through RAG.

Checklist Before You Call This Done

- ☐ 3 independent reviewer agents (Security, Style, Test-Coverage), each tested on its own before joining the team
- ☐ The Style reviewer is grounded in a real, searched style-guide document, not general model knowledge
- ☐ The Supervisor routes based on the diff — not every reviewer runs on every diff
- ☐ The final output is one compiled, cleaned-up, sorted review — not three reports stuck together
- ☐ You designed this project's own step breakdown, design document, and file layout, without them being handed to you in full detail

Status

Not started. Track your progress in [../PROGRESS.md](#).

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say "**Show me the solution.**" — but try harder before asking here than you did on Projects 1-5. That's the actual exercise.